

# Practice 16 详解答案

## A. 单项选择题

1. **B**: NoteManager 负责管理和协调多条笔记。
2. **B**: 管理器拥有笔记集合, 是 has-a 的组合关系。
3. **B**: 集中集合级职责, 使程序结构更清晰。
4. **B**: `vector<Note> notes` 表示管理器保存多个笔记。
5. **C**: 单条 Note 应负责显示自己的内容。
6. **B**: 委托是把任务交给最适合的对象。
7. **A**: 单一职责原则要求类聚焦于一个主要职责。
8. **B**: 管理器协调专门对象更利于扩展。
9. **B**: 高内聚表示类中的职责紧密相关。
10. **A**: 从单个类设计提升到多个类协作的架构设计。

## B. 填空题

1. 管理 (拥有)
2. has
3. 合适
4. 高
5. 耦合

## C. 判断题

1. **错**: 管理器应处理集合协调, 不应吸收每个对象的细节逻辑。
2. **对**: 组合表示一个对象拥有或包含其他对象。
3. **对**: 管理器管理集合, 笔记管理自己的状态和单条显示。
4. **错**: 高耦合会使修改一个类更容易影响其他类。
5. **对**: 职责集中时, 测试和理解都更直接。
6. **对**: `main()` 应协调程序流程, 而不是保存所有业务细节。
7. **对**: has-a 关系通常使用成员对象或容器表达。
8. **错**: `vector<Note>` 表示组合, 不是继承。
9. **对**: 委托让一个职责只实现一次, 其他对象调用它。
10. **对**: 职责边界清楚时, 新功能影响范围更小。

## D. 代码阅读

### 1. NoteManager 与 Note 的关系

```
vector<Note> notes;
```

这是\*\*组合 (has-a) \*\*关系：一个 NoteManager 拥有并管理多个 Note 对象，不是 NoteManager is-a Note 的继承关系。

### 2. main() 有 500 行代码的问题

职责混杂：菜单、输入验证、容器管理、搜索、删除和显示都挤在同一处。代码难读、难测、难改；任何功能变更都可能影响庞大的 main()。应把集合操作提取到 NoteManager，单条笔记行为放入 Note。

### 3. 展示的设计原则

```
manager.addNote(note);
```

这体现了**封装和委托**。main() 不直接知道容器如何保存笔记，而把“添加到集合”的职责委托给 NoteManager。

### 4. 为什么这个设计更好

Note::display() 只显示自身，是单对象职责；NoteManager::displayAll() 遍历集合并委托每个 Note 显示自己，是集合职责。职责清晰，后续可单独修改笔记格式或集合遍历方式。

### 5. 展示的设计原则

manager.displayAll() 再调用每条笔记的 display()，展示了**委托和单一职责原则**：管理器负责协调，笔记负责展示自身。

## E. 简答题

### 1. 为什么使用 NoteManager

直接在 main() 中保存 vector<Note> 会让主流程知道太多集合细节。NoteManager 将添加、删除、搜索、显示等操作集中在一个受控接口中，隐藏容器实现。main() 只处理菜单与流程，代码更易测试，未来可将 vector 改为文件或数据库而减少外部影响。

## 2. 继承与组合的区别

继承表达 is-a: `TextNote` 是 `Note`，可用基类接口代替。组合表达 has-a: `NoteManager` 有一组 `Note`。继承主要用于类型层次与多态；组合用于组织对象之间的拥有或协作关系。不能把“管理器是一条笔记”误建模为继承。

## 3. 高内聚、低耦合为何重要

高内聚让一个类的成员都服务于同一职责，例如 `NoteManager` 的成员都与笔记集合有关。低耦合让类只依赖必要的公开接口，不依赖别的类内部细节。两者使改动范围小、单元测试容易、复用性更高。

## F. 编程题

### 1. 简单图书管理器

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;

class Book
{
private:
    string title;

public:
    Book(const string& bookTitle) : title(bookTitle) {}

    void display() const
    {
        cout << "Title: " << title << '\n';
    }
};

class Library
{
private:
    vector<Book> books;

public:
    void addBook(const Book& book)
    {
        books.push_back(book);
    }

    void displayAll() const
    {
        for (const Book& book : books) {
            book.display();
        }
    }
};

int main()
{
    Library library;
```

```
library.addBook(Book("Clean Code"));
library.addBook(Book("C++ Primer"));
library.displayAll();
}
```

## 2. 在集合中搜索

给 Note 提供只读的标题匹配操作，再由 NoteManager 遍历集合：

```
class Note
{
private:
    string title;

public:
    Note(const string& noteTitle) : title(noteTitle) {}

    bool hasTitle(const string& searchedTitle) const
    {
        return title == searchedTitle;
    }
};

class NoteManager
{
private:
    vector<Note> notes;

public:
    void addNote(const Note& note)
    {
        notes.push_back(note);
    }

    bool containsTitle(const string& title) const
    {
        for (const Note& note : notes) {
            if (note.hasTitle(title)) {
                return true;
            }
        }
        return false;
    }
};
```

返回 `true` 表示至少存在一条标题完全相同的笔记。

### 3. 删除笔记

```
#include <algorithm>
#include <iostream>

void removeNote(const string& title)
{
    vector<Note>::iterator found = find_if(
        notes.begin(), notes.end(),
        [&title](const Note& note) {
            return note.hasTitle(title);
        });

    if (found == notes.end()) {
        cout << "Note not found.\n";
        return;
    }

    notes.erase(found);
    cout << "Note removed.\n";
}
```

这段成员函数应写在 `NoteManager` 内部；它删除第一个标题匹配的元素。需要包含 `<algorithm>`，`notes` 是管理器的私有 `vector<Note>`。

### 4. 分离职责

建议重构为：

```
class Note
{
public:
    void display() const;
    void saveToFile(const string& filename) const;
    bool contains(const string& keyword) const;
};

class NoteManager
{
public:
    void addNote(const Note& note);
    void removeNote(const string& title);
    void search(const string& keyword) const;
    void sortByTitle();
    void loadAll(const string& filename);
    void backup(const string& filename) const;
};
```

`Note` 保留单条笔记的显示、保存和内容匹配。`NoteManager` 负责多条笔记的搜索、排序、删除、批量加载和备份。这样单条逻辑不会知道集合细节，集合逻辑也不必直接暴露每条笔记的内部数据。

## 5. CLI Notes 架构（挑战）

```
#include <algorithm>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

using namespace std;

class Note
{
private:
    string title;
    string content;

public:
    Note(const string& noteTitle, const string& noteContent)
        : title(noteTitle), content(noteContent) {}

    bool hasTitle(const string& searchedTitle) const
    {
        return title == searchedTitle;
    }

    bool contains(const string& keyword) const
    {
        return title.find(keyword) != string::npos ||
            content.find(keyword) != string::npos;
    }

    void display() const
    {
        cout << "Title: " << title << '\n';
        cout << "Content: " << content << "\n\n";
    }
};

class NoteManager
{
private:
    vector<Note> notes;

public:
    void addNote(const Note& note)
    {

```



```

        notes.push_back(note);
        cout << "Note added.\n";
    }

    void removeNote(const string& title)
    {
        vector<Note>::iterator found = find_if(
            notes.begin(), notes.end(),
            [&title](const Note& note) { return note.hasTitle(title); });

        if (found == notes.end()) {
            cout << "Note not found.\n";
            return;
        }

        notes.erase(found);
        cout << "Note removed.\n";
    }

    void search(const string& keyword) const
    {
        bool found = false;
        for (const Note& note : notes) {
            if (note.contains(keyword)) {
                note.display();
                found = true;
            }
        }

        if (!found) {
            cout << "No matching notes.\n";
        }
    }

    void displayAll() const
    {
        if (notes.empty()) {
            cout << "No notes available.\n";
            return;
        }

        for (const Note& note : notes) {
            note.display();
        }
    }
};

```

```

int main()
{
    NoteManager manager;
    int choice = 0;

    while (choice != 5) {
        cout << "1. Add\n2. Remove\n3. Search\n4. Display all\n5. Exit\n";
        cout << "Choose: ";

        if (!(cin >> choice)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Please enter a number.\n";
            continue;
        }
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        string title;
        string content;
        string keyword;

        switch (choice) {
            case 1:
                cout << "Title: ";
                getline(cin, title);
                cout << "Content: ";
                getline(cin, content);
                manager.addNote(Note(title, content));
                break;
            case 2:
                cout << "Title to remove: ";
                getline(cin, title);
                manager.removeNote(title);
                break;
            case 3:
                cout << "Search keyword: ";
                getline(cin, keyword);
                manager.search(keyword);
                break;
            case 4:
                manager.displayAll();
                break;
            case 5:
                cout << "Goodbye.\n";
                break;
        }
    }
}

```

```
        default:
            cout << "Invalid choice.\n";
        }
    }
}
```

notes 始终是 NoteManager 的 private 成员，main() 只能经由 addNote、removeNote、search 和 displayAll 操作系统。

## G. 设计与思考挑战

设计 B 更容易维护。NoteManager 封装了集合和所有集合级操作，main() 只负责程序流程。新增排序、持久化或筛选功能时，可在管理器中实现和测试，不必继续膨胀 main()；也能更换内部容器而不改变主菜单代码。设计 A 使每项功能直接依赖 main() 里的容器和局部变量，程序增长后耦合更高。